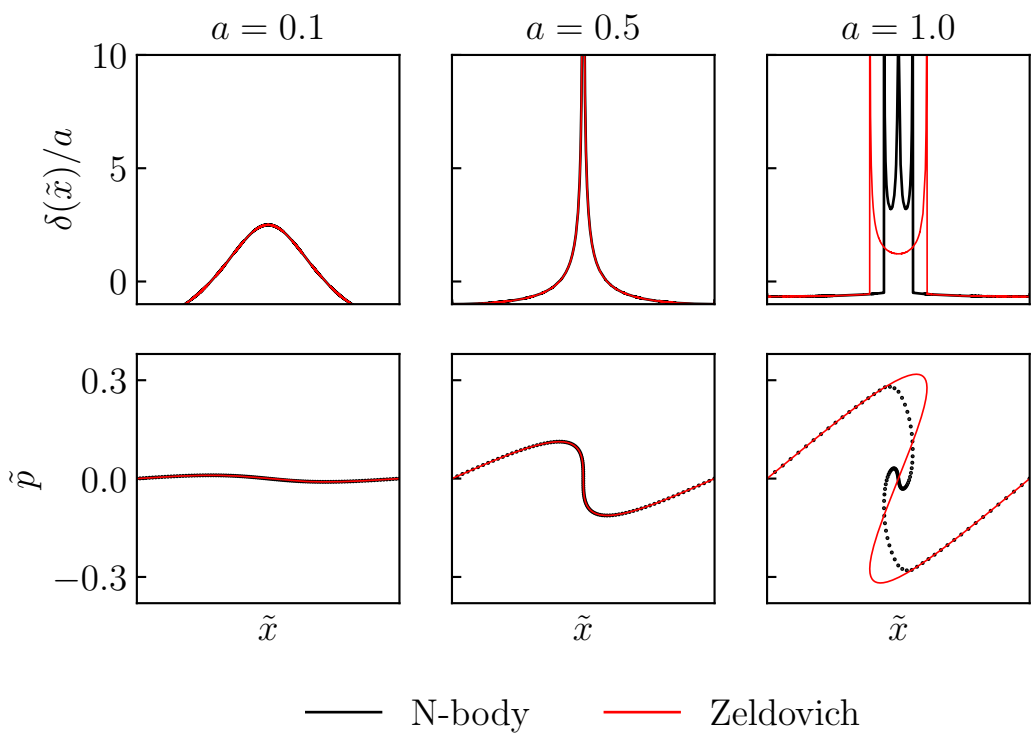
Boltzmann Code - N-body (Zeldovich Pancake)

The goal of today's lab is to take a short detour from our Einstein-Boltzmann code and write a tiny cosmological N-body code from scratch. In particular, we will look at the collapse of a single plane wave in a matter-only ($\Omega_m = 1$) universe using a **particle-mesh** (PM) scheme[1]. At the end of this lab you should be able to recreate the following plot, which compares your N-body solution to the analytic Zeldovich prediction (see also `sol/figures/section04_pancake.mp4`):



[1] You can actually do 1-D sims very easily without a mesh just by counting the number of particles to the left and right of you to get the force on you (think about the electric field of an infinite plane). However I'll have you write a particle mesh code because that's what we do 3-D which is what we actually care about.

Assignment: The equations governing a system of (collisionless) dark matter particles is very simple:

$$\nabla_r^2 \Phi = 4\pi G\rho, \qquad \frac{d\boldsymbol{r}}{dt} = \boldsymbol{u}, \qquad \frac{d\boldsymbol{u}}{dt} = -\nabla_r \Phi. \tag{1}$$

This is fine in principle but inconvenient in practice because the universe is expanding. Really, we'd like our simulation box to stay a fixed size. So let's switch to comoving coordinates $\boldsymbol{x} = \boldsymbol{r}/a$ and define the peculiar velocity through

$$\dot{\boldsymbol{r}} = \boldsymbol{u} = a\dot{\boldsymbol{x}} + \dot{a}\boldsymbol{x} \equiv \boldsymbol{v} + H\boldsymbol{r} = \text{Peculiar Velocity} + \text{Hubble Flow}.$$

In particular we've defined $\boldsymbol{v} = a\dot{\boldsymbol{x}} = \boldsymbol{u} - H\boldsymbol{r}$. This then leads us to define the comoving momentum $\boldsymbol{p} = a\boldsymbol{v}$ and **peculiar potential**

$$\phi \equiv \Phi + \tfrac{1}{2}a\ddot{a}\boldsymbol{x}^2.$$

You can (and should) work out that in terms of these new variables our system of equations Eq.(1) becomes

$$\nabla^2 \phi = 4\pi G a^2 \bar{\rho}\delta, \qquad \frac{d\boldsymbol{x}}{da} = \frac{\boldsymbol{p}}{\dot{a}a^2}, \qquad \frac{d\boldsymbol{p}}{da} = -\frac{\nabla\phi}{\dot{a}}, \tag{2}$$

where now ∇ is taken with respect to $\boldsymbol{x}$ and we've traded $t \to a$ as the time variable using $dt = da/\dot{a}$. It is a good idea numerically to evolve dimensionless quantities (code units). Variables in code units will carry a tilde. We will have

$$\tilde{\boldsymbol{x}} = \frac{1}{a}\frac{\boldsymbol{r}}{r_0}, \quad \tilde{\boldsymbol{p}} = a\frac{\boldsymbol{v}}{v_0}, \quad \tilde{\phi} = \frac{\phi}{\phi_0}, \quad \tilde{\rho} = a^3\frac{\rho}{\rho_0},$$

with $r_0 = L_{\text{box}}$, $t_0 = 1/H_0$, $v_0 = r_0/t_0$, $\phi_0 = v_0^2$, $\rho_0 = 3H_0^2/(8\pi G)$. Once again you can (should) show that in terms of these dimensionless variables, the

system of equations becomes

$$\boxed{\frac{d\tilde{\boldsymbol{x}}}{da} = \frac{\tilde{\boldsymbol{p}}}{a^{3/2}}, \qquad \frac{d\tilde{\boldsymbol{p}}}{da} = -\sqrt{a}\tilde{\nabla}\tilde{\phi}, \qquad \tilde{\nabla}^2\tilde{\phi} = \frac{3}{2a}\tilde{\delta},} \tag{3}$$

where $\tilde{\nabla} = \partial/\partial\tilde{x}_i$ and $\tilde{\delta} = \tilde{\rho} - 1$. These are the equations we'll actually solve.

The recipe for evolving Eq.(3) from initial conditions $\{\tilde{x}_i, \tilde{p}_i\}$ has three pieces:

- Given particle positions $\{\tilde{x}_i\}$, compute the density $\tilde{\rho}$ on a grid (**Part 1**).
- Given $\tilde{\rho}$ on the grid, solve the Poisson equation for $\tilde{\phi}$ (**Part 2**).
- Given $\tilde{\phi}$, compute the force at each particle and evolve (**Part 3**).

Density from particles: We have N_p particles. We want to use these particles to estimate the mass density on a grid of N_g cells. Assume all particles have the same mass $m = 1/N_p$. The simplest density-assignment scheme is **nearest grid point** where you dump all of a particle's mass in the cell that contains it. A better choice, and the one we'll use throughout, is the **Cloud-in-Cell** (CIC) scheme where you treat each particle as a uniform-density block of size Δx, and its mass is split between the two cells that its block overlaps. If the particle sits at $\tilde{x}$ and its left edge is at $\tilde{x}_L = \tilde{x} - \frac{1}{2}\Delta x$, then the cell containing $\tilde{x}_L$ gets a fraction[2]

$$\text{frac} = 1 + \lfloor \tilde{x}_L/\Delta x \rfloor - \tilde{x}_L/\Delta x$$

of the particle's mass and the next cell to the right gets the remaining $1 - \text{frac}$ (with periodic wrapping at the edges). Doing this for all particles gives you a mass distribution which then becomes a density through multiplying by $1/\Delta x = N_g$. As a sanity check, if particles are uniformly distributed then $\tilde{\rho} \to 1$ in every cell.

[2]Work through a couple of simple cases on paper if the definition is not obvious. This definition lets you vectorize the CIC deposit.

We also need the inverse operation. Given some quantity sampled at cell centers (e.g. $\tilde{\nabla}\tilde{\phi}$), what is its value at the particle? It is very important that we use the same shape for this interpolation as we used for the density assignment.

Part 1: Write a function `CIC_deposit(x, N_grid)` that takes a 1D array of particle positions $x \in [0, 1)$ and returns the density $\tilde{\rho}$ as a array of length `N_grid`. A vectorized implementation using `np.bincount` (with two calls, one for each cell receiving a fraction of mass) is very fast.

Then write `CIC_interpolate(x, grad_phi)`, which takes a quantity sampled at the centers of the same grid and returns its CIC-interpolated value at each particle position. Use exactly the same fractional weights as in `CIC_deposit`.

Part 2: Solving the Poisson equation with FFT: With $\tilde{\rho}$ in hand, our task is to solve

$$\tilde{\nabla}^2\tilde{\phi} = \frac{3}{2a}\tilde{\delta}, \qquad \tilde{\delta} = \tilde{\rho} - 1.$$

We will approximate the Laplacian with finite differences. Let μ index the grid-cells (where we've been using i to index particles)

$$\tilde{\nabla}^2\tilde{\phi}_\mu \approx N_g^2\big(\tilde{\phi}_{\mu-1} + \tilde{\phi}_{\mu+1} - 2\tilde{\phi}_\mu\big) = \frac{3}{2a}\tilde{\delta}_\mu.$$

In doing this, we basically have reduced solving the Poisson equation to a system of N_{grid} linear equations. As with all linear things, we can find it's Green's function (see `lecture_notes/section04.pdf`)

$$\boxed{\tilde{\phi}(k_\ell) = -\frac{3}{8aN_g^2} \times \frac{1}{\sin^2(k_\ell/2)} \times \tilde{\delta}_\ell} \tag{4}$$

You'll have to manually set $\tilde{\phi}(k_\ell = 0) = 0$ (a uniform shift up or down to

the potential shouldn't be observable anyways). The inverse FFT of the above gives us our solution for $\tilde{\phi}_\mu$.

Part 2: Write a function `solve_poisson(rho, a)` that uses `np.fft` to compute $\tilde{\phi}_\mu$ given the input density `rho` (ρ_μ). Note that the wavenumbers from `np.fft.fftfreq` are ℓ/N_g, so multiply by 2π to get k_ℓ.

Once we've got $\tilde{\phi}_\mu$, we still need its gradient at each particle. Compute $\tilde{\nabla}\tilde{\phi}$ on the grid first using finite difference,

$$\tilde{g}_\mu = \frac{\tilde{\phi}_{\mu+1} - \tilde{\phi}_{\mu-1}}{2\Delta x} = \frac{N_g}{2}\big(\tilde{\phi}_{\mu+1} - \tilde{\phi}_{\mu-1}\big),$$

and then CIC-interpolate (using the function from Part 1) back to the particles to get $\tilde{g}_i$. Use `np.roll` for the central difference so the periodic boundaries take care of themselves.

Evolution (kick-drift-kick): For the time integrator we'll use second-order **leapfrog**. Specifically, something called kick-drift-kick (KDK). Given $\{\tilde{x}_i^{(n)}, \tilde{p}_i^{(n)}\}$ at a_n, we can evolve the system by Δa through

- **Kicking** momentum $\Delta a/2$: use the current density to solve for $\tilde{\phi}$, compute $\tilde{g}_i$ at each particle, and update

$$\tilde{p}_i^{(n+1/2)} = \tilde{p}_i^{(n)} - \sqrt{a_n} \times \tilde{g}_i^{(n)} \frac{\Delta a}{2}.$$

- **Drifting** positions Δa: use the half-step momentum, with the drift coefficient evaluated at the midpoint $a_{n+1/2} = a_n + \Delta a/2$,

$$\tilde{x}_i^{(n+1)} = \tilde{x}_i^{(n)} + a_{n+1/2}^{-3/2}\, \tilde{p}_i^{(n+1/2)}\, \Delta a,$$

- **Kicking** momentum another $\Delta a/2$: re-deposit the density at $\tilde{x}^{(n+1)}$, resolve Poisson, and finish updating the momentum with the kick coefficient

at $a_{n+1} = a_n + \Delta a$,

$$\tilde{p}_i^{(n+1)} = \tilde{p}_i^{(n+1/2)} - \sqrt{a_{n+1}} \times \tilde{g}_i^{(n+1)} \frac{\Delta a}{2}.$$

Evaluating the time-dependent coefficients $\sqrt{a}$ and $a^{-3/2}$ at the midpoint of each substep is what makes the scheme globally second-order accurate in Δa.

Initial conditions (Zeldovich pancake): For the test problem we'll seed our particles with a single sine-wave displacement and evolve. This is the canonical 1D test for any cosmological N-body code because it's easy to check with the **Zeldovich approximation** which is exact in 1D until shell-crossing. If you haven't seen this before (I think Oliver will cover it at some point?) take a look at `aux/HW07*`. For a wave of comoving wavelength equal to the box size,

$$\tilde{x}(q, a) = q + aA \sin(2\pi q), \qquad \tilde{p}(q, a) = a^{3/2} A \sin(2\pi q),$$

where $q \in [0, 1]$ is the Lagrangian (initial) coordinate and the amplitude $A = [a_{\text{cross}} \cdot 2\pi]^{-1}$ is fixed by demanding that the first caustic forms at $a = a_{\text{cross}}$.[3] The momentum here just follows from the equation of motion in Eq.(3).

Part 3: Write `kick(x, p, a, da, N_grid)` and `drift(x, p, a, da)` that apply the KDK steps above. Then evolve $N_p \sim 10^5$ particles initialized via Zeldovich with $a_{\text{init}} = 0.1$ and $a_{\text{cross}} = 0.5$ from a_{init} out to $a_{\text{final}} = 1$ with $\Delta a = 10^{-4}$ on a grid with $N_g \sim 10^4$ cells.

[3] The density follows from mass conservation $\tilde{\rho} d\tilde{x} = dq$, giving $\tilde{\rho}(q, a) = [1 + aA \cdot 2\pi \cos(2\pi q)]^{-1}$, which diverges first at $q = 1/2$ when $aA \cdot 2\pi = 1$, hence the choice of A.

Validation: Pick three snapshots, say $a \in \{0.1, 0.5, 1.0\}$, and overlay the N-body $\tilde{\delta}(\tilde{x})/a$ against the Zeldovich prediction. You should also compare the phase-space $(\tilde{x}, \tilde{p})$ from your simulation against the Zeldovich prediction.

If everything went well, you should see the same picture as on the first page of this handout.

- At $a = 0.1$ the wave is barely perturbed and the two solutions agree.
- At $a = a_{\text{cross}} = 0.5$ the analytic density diverges at $\tilde{x} = 1/2$ and the N-body has a tall narrow spike there.
- At $a = 1.0$ the system is well past shell crossing where the analytic Zeldovich prediction breaks down. You start form a phase-space spiral in the DM.

And with this, you now know the basics of N-body simulation, there's nothing conceptually new when you generalize particle mesh schemes (what we've done here) to 3-D and to ΛCDM. The only thing we're missing is a cosmological initial condition. This requires (in the standard cosmological model) generating a Gaussian random field whose power spectrum matches what we expect from evolving the primordial inflationary perturbation through recombination (what we will get out of our Einstein-Boltzmann code!) Hopefully we'll have time to revisit this in section at the end of the quarter (if you're curious, see the references).

References:

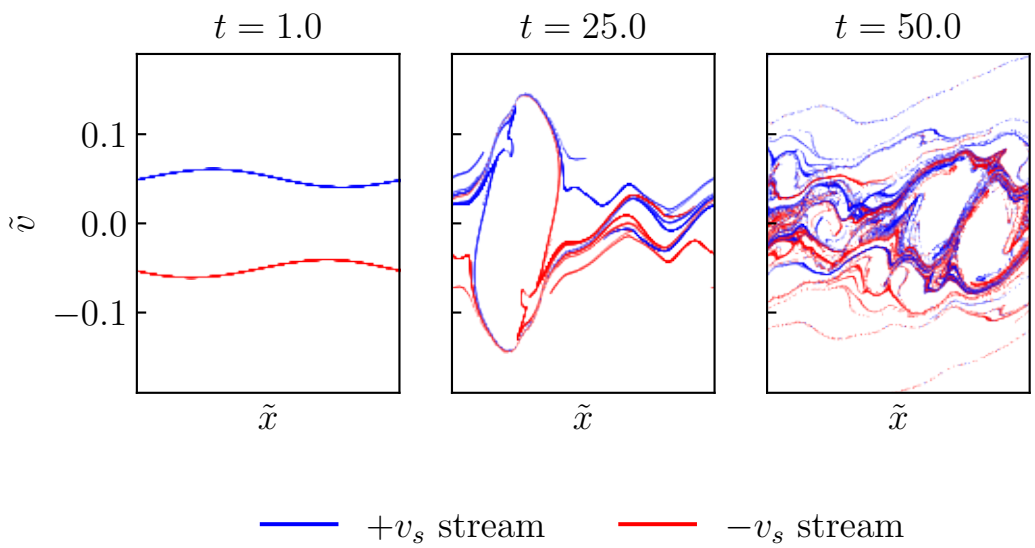
- Kravtsov (2002), *Writing a PM code*, notes from a 2002 lecture at U. Chicago.
  This handout is essentially a 1D, $\Omega_m = 1$ version of Kravtsov's notes. You should read his notes for the full story. The notes also include a section on setting up realistic cosmological initial conditions from a power spectrum, which is the natural thing to do once your pancake looks good.

- *My old PM handouts*, github.com/DelonShen/pm_handouts.
  Some old notes + code I wrote on particle mesh. Handout 2 includes a discussion on setting up cosmological initial conditions from a power spectrum.

- Abel, Hahn & Kaehler (2012), *Tracing the dark matter sheet in phase space*, arXiv:1111.3944.
  There is a "right" way to think about density assignment which is to track the 3-D dark-matter sheet in 6-D phase space rather than treating particles as point masses. This leads to significantly less noise.

- McQuinn & White (2016), *Cosmological perturbation theory in* $1 + 1$ *dimensions*, arXiv:1502.07389.
  A cute example of some work which uses the toy 1-D simulation you just built. My old PM handouts were written partially to help others reproduce the results of this paper (you need to compare all that EFT to something).

(Bonus) Two-stream instability (PIC): Secretly what you've written here is a **very** simple plasma simulation. The particle-in-cell codes / kinetic plasma simulations you hear about from your plasma friends at their core do the exact same things as the N-body code we've written here (they do have a much harder time though with numerics, magnetic fields, relativistic velocities, ...). So, with essentially the exact same code you've written (remove expansion and replace mass with charge) you can look at classic plasma phenomena. As an example, I've shown[4] below a classic two-stream instability (start with two cold beams of electrons. one moving to the right and one moving to the left).



See also `sol/figures/section04_two_stream.mp4`.

[4] I have really not checked in detail for numerical well-behavedness here